

☁ QB 09: Cloud Computing & DevOps — Question Bank with Answer Keys

Course: Cloud Computing / DevOps & Cloud Infrastructure (Elective / Core)

Target: VTU MCA Semester 3 + Student DevSecOps Career Path

Scheme: VTU 2022 / 2024 Scheme

Contents: NIST Cloud Models, Docker vs VMs, Kubernetes Cluster Architecture, CI/CD Pipelines, Terraform IaC, DevSecOps "Shift Left", 15 MCQs with Explanations

Syllabus Outline (VTU 5 Modules)

- **Module 1:** Cloud Computing Fundamentals, NIST Definition, 5 Essential Characteristics, Service Models (IaaS, PaaS, SaaS, FaaS), Deployment Models (Public, Private, Hybrid).
- **Module 2:** Virtualization vs Containerization, Hypervisors (Type 1 vs Type 2), Docker Architecture, Dockerfiles, Image Layers, Container Networking & Storage.
- **Module 3:** Container Orchestration with Kubernetes: Control Plane Components (API Server, etcd, Scheduler, Kube-controller), Worker Nodes (Kubelet, Kube-proxy), Pods, Services, Deployments, Ingress.
- **Module 4:** DevOps Culture & CI/CD: Agile vs DevOps, Continuous Integration, Continuous Delivery vs Continuous Deployment, Pipeline Stages, GitHub Actions, Jenkins.
- **Module 5:** Infrastructure as Code (IaC) & DevSecOps: Terraform, Declarative vs Imperative Configuration, "Shift Left" Security, SAST, DAST, Container Scanning (Trivy), Monitoring (Prometheus & Grafana).

SECTION 1: High-Yield MCQs with Answer Key

Q1. Which of the following is NOT one of the 5 NIST essential characteristics of Cloud Computing?

- A) On-demand self-service
- B) Broad network access
- C) Fixed hardware ownership
- D) Rapid elasticity

Answer: C

Explanation: Cloud computing eliminates fixed hardware ownership. The 5 NIST characteristics are: (1) On-demand self-service, (2) Broad network access, (3) Resource pooling, (4) Rapid elasticity, and (5) Measured service.

Q2. In which cloud service model does the cloud provider manage everything up through the runtime and middleware, leaving the customer to manage only the application code and data?

- A) IaaS (Infrastructure as a Service)
- B) PaaS (Platform as a Service)
- C) SaaS (Software as a Service)
- D) On-Premises

Answer: B

Explanation: In PaaS (e.g., AWS Elastic Beanstalk, Heroku, Google App Engine), the OS, runtime, and middleware are managed by the provider; developers manage only application logic and data.

Q3. What is the fundamental difference between a Virtual Machine (VM) and a Docker Container?

- A) VMs share the host OS kernel; containers have their own guest OS
- B) Containers share the host OS kernel and isolate user spaces; VMs virtualize physical hardware and require a complete guest OS
- C) Containers are slower to boot than VMs
- D) VMs use less memory than containers

Answer: B

Explanation: Containers share the underlying host OS kernel, making them lightweight (MBs, sub-second boot time) compared to heavy VMs requiring hypervisors and full guest OS installs (GBs).

Q4. Which component of the Kubernetes Control Plane acts as the single source of truth and distributed key-value datastore?

- A) kube-apiserver
- B) kube-scheduler
- C) etcd
- D) kubelet

Answer: C

Explanation: etcd is a highly available, consistent, distributed key-value store used to persist all cluster state and configurations in Kubernetes.

Q5. What is the difference between CMD and ENTRYPOINT in a Dockerfile?

- A) CMD cannot be overridden from the CLI
- B) ENTRYPOINT defines the base executable command; CMD provides default arguments that can be easily overridden at runtime via docker run
- C) ENTRYPOINT only runs during image build
- D) There is no difference

Answer: B

Explanation: `ENTRYPOINT` sets the fixed process to run. `CMD` supplies default arguments that are appended to the entrypoint unless overridden by command-line parameters.

Q6. In Kubernetes, which Service type provides a static public IP address accessible directly from external internet traffic?

- A) `ClusterIP`
- B) `NodePort`
- C) `LoadBalancer`
- D) `Headless`

Answer: C

Explanation: `LoadBalancer` provisions an external cloud provider load balancer (e.g., AWS Network Load Balancer) to route public internet traffic into the cluster.

Q7. What does "Shift Left" mean in modern DevSecOps?

- A) Postponing security testing until after deployment to production
- B) Integrating security scanning, code analysis, and compliance early in the development and CI/CD lifecycle
- C) Moving servers to left-side data racks
- D) Writing code from right to left

Answer: B

Explanation: "Shift Left" integrates security audits (SAST, secret scanning, dependency checks) directly into the developer workflow and build pipeline, catching vulnerabilities before code ever reaches production.

Q8. What file does Terraform use to track the current mapped state of real-world cloud infrastructure?

- A) `main.tf`
- B) `terraform.tfstate`
- C) `variables.tf`
- D) `outputs.tf`

Answer: B

Explanation: `terraform.tfstate` maps declarative resource definitions to real-world cloud resources, tracking metadata and resource IDs.

Q9. What is the difference between Continuous Delivery and Continuous Deployment?

- A) Continuous Delivery requires manual approval before releasing to production; Continuous Deployment deploys automatically without human intervention
- B) Continuous Delivery does not run automated tests
- C) Continuous Deployment is only for Docker containers

- D) Continuous Delivery only deploys on weekends

Answer: A

Explanation: In Continuous Delivery, software is automatically built and tested to be production-ready at all times, but actual production release requires a manual trigger. In Continuous Deployment, every passing build deploys to production automatically.

Q10. Which tool performs Static Application Security Testing (SAST) by inspecting source code for vulnerabilities without executing it?

- A) OWASP ZAP
- B) SonarQube
- C) Wireshark
- D) Nmap

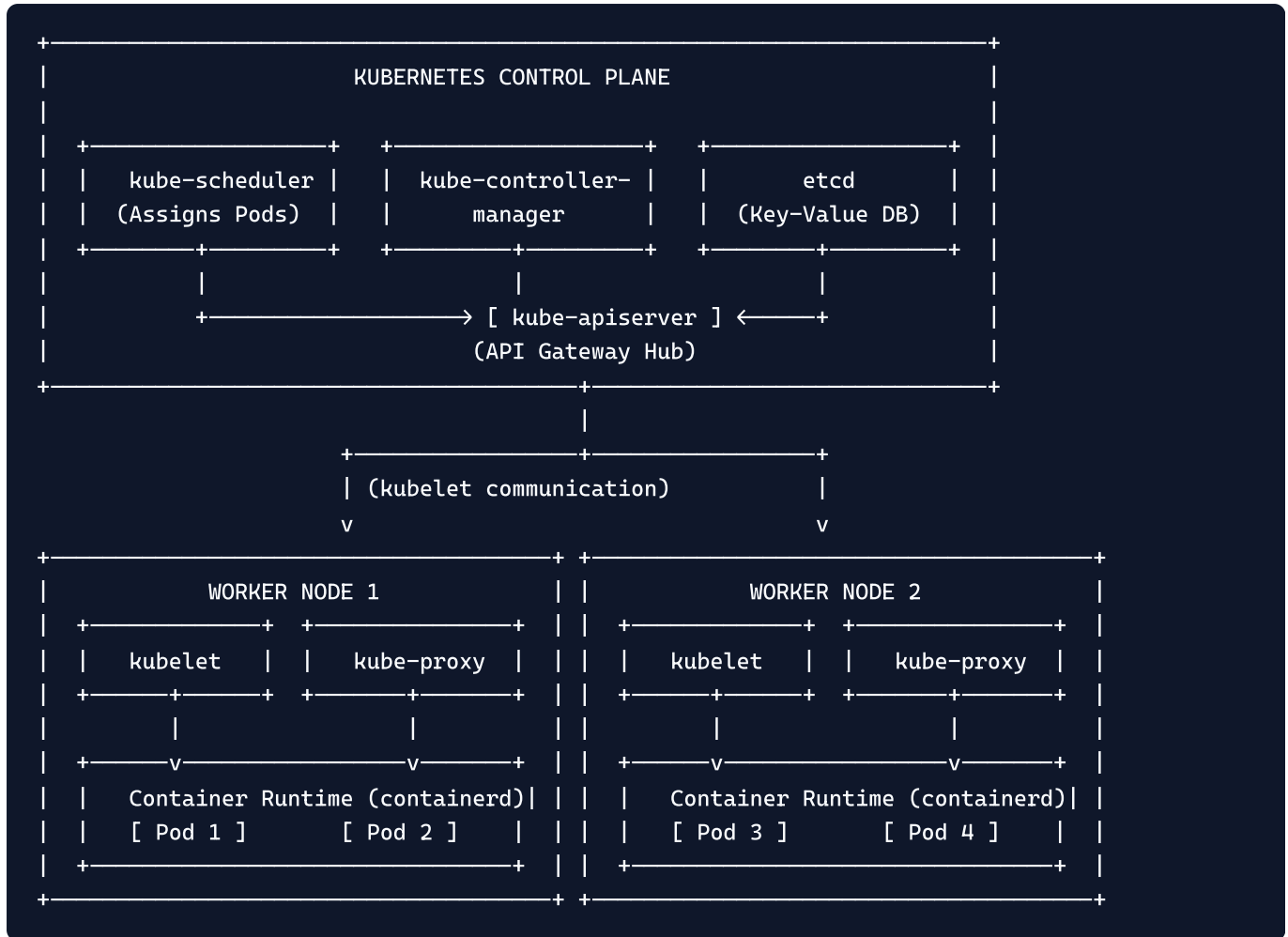
Answer: B

Explanation: SonarQube analyzes source code statically to identify bugs, code smells, and security vulnerabilities without running the program. (OWASP ZAP is DAST).

 SECTION 2: Short Answer Concepts (4–6 Marks)

Q11. Explain Kubernetes Architecture with Control Plane and Worker Node components.

Answer:



Key Components:

1. Control Plane (Master Node):

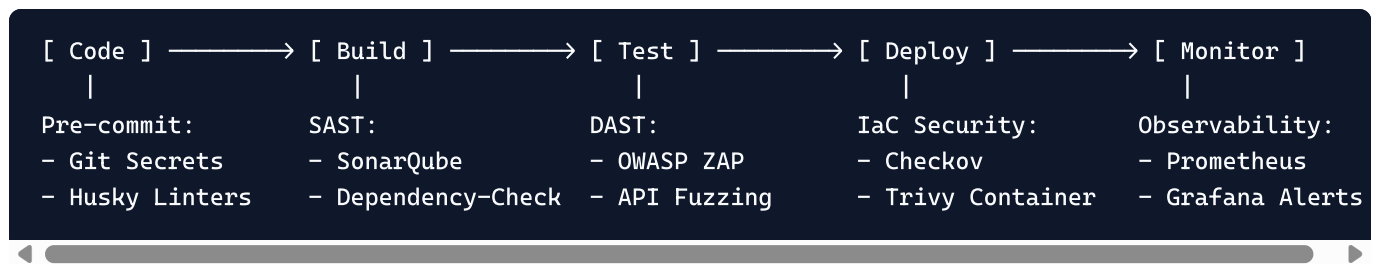
- **kube-apiserver** : Exposes the Kubernetes HTTP REST API; central communication gateway.
- **etcd** : Consistent, distributed database storing cluster configuration and state.
- **kube-scheduler** : Selects the best worker node for newly created pods based on resource availability.
- **kube-controller-manager** : Runs daemon controllers (Node Controller, Replication Controller, Endpoint Controller).

2. Worker Nodes:

- **kubelet** : Agent running on each node ensuring that containers described in PodSpecs are running and healthy.
- **kube-proxy** : Network proxy managing IP routing, iptables, and load-balancing traffic across pods.
- **Container Runtime (containerd /CRI-O)**: Software responsible for running containers.

Q12. Describe the DevSecOps "Shift Left" Pipeline Stages.

Answer: Traditional DevOps integrated security as a final gatekeeper step, creating bottlenecks. **DevSecOps** embeds automated security checks into every phase of the software development lifecycle (SDLC):



1. **Pre-Commit (IDE/Developer machine):** Secret detection (`git-secrets` , Gitleaks) prevents accidental credential commits.
2. **Commit / Build (CI): SAST** (Static Analysis) scans code for injection flaws; SCA (Software Composition Analysis) scans dependencies for known CVEs.
3. **Test Phase: DAST** (Dynamic Analysis) runs automated penetration tests against running staging containers.
4. **Deploy Phase:** Scans Terraform configurations (`tfsec` / `Checkov`) for security misconfigurations (e.g., unencrypted S3 buckets, open 0.0.0.0/0 security groups).
5. **Runtime / Monitor:** Runtime protection and monitoring (Falco, Prometheus, CloudWatch).

SECTION 3: VTU Model Practical Architecture & Scripts (10–12 Marks)

Q13. [VTU Model QP - Production Multi-Stage Dockerfile]

Write a production-grade, secure Multi-Stage Dockerfile for a Node.js web application. Explain why multi-stage builds are critical for container security and size optimization.

Answer:

```
# =====  
# STAGE 1: Build & Dependency Stage  
# =====  
FROM node:20-alpine AS builder  
  
# Set working directory  
WORKDIR /app  
  
# Copy dependency manifests first (Leverages Docker build cache)  
COPY package*.json ./  
  
# Install dependencies (including devDependencies for TypeScript/tests)  
RUN npm ci  
  
# Copy application source code  
COPY . .  
  
# Compile application (e.g., TypeScript to JS bundle)  
RUN npm run build  
  
# Remove development dependencies to prune node_modules  
RUN npm prune --production  
  
# =====  
# STAGE 2: Minimal Production Runtime  
# =====  
FROM node:20-alpine AS runner  
  
WORKDIR /app  
  
# SECURITY: Create a non-root dedicated system user and group  
RUN addgroup -S appgroup && adduser -S appuser -G appgroup  
  
# Set production environment  
ENV NODE_ENV=production  
ENV PORT=3000  
  
# Copy only the compiled output and pruned production modules from builder  
COPY --from=builder /app/node_modules ./node_modules  
COPY --from=builder /app/dist ./dist  
COPY --from=builder /app/package.json ./package.json  
  
# Assign ownership to the non-root user  
USER appuser  
  
# Expose application port  
EXPOSE 3000  
  
# Define container startup command  
CMD ["node", "dist/server.js"]
```

Why Multi-Stage Builds are Critical for Security & Efficiency:

1. **Dramatically Smaller Image Size:** Development tools (compilers, TypeScript transpilers, git, devDependencies) are discarded. Image size drops from ~1.2 GB to ~120 MB.
 2. **Minimal Attack Surface:** Fewer installed binaries and packages mean drastically fewer CVE vulnerabilities that an attacker can exploit.
 3. **Non-Root Execution:** Running as `USER appuser` prevents container breakout attacks from gaining root control of the underlying host kernel.
-

Q14. [VTU Model QP - GitHub Actions CI/CD Pipeline Workflow]

Write a complete GitHub Actions YAML workflow (`.github/workflows/deploy.yml`) that triggers on every push to the `main` branch, runs automated unit tests, scans the Docker image for vulnerabilities using Trivy, and pushes the image to Docker Hub upon success.

Answer:


```
name: DevSecOps CI/CD Pipeline

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build-test-and-scan:
    name: Build, Security Scan & Push
    runs-on: ubuntu-latest

    steps:
      # Step 1: Check out source code repository
      - name: Checkout Code
        uses: actions/checkout@v4

      # Step 2: Setup Node.js runtime environment
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      # Step 3: Install dependencies & run unit tests
      - name: Install & Run Unit Tests
        run: |
          npm ci
          npm test

      # Step 4: Build Docker Image locally for scanning
      - name: Build Local Docker Image
        run: |
          docker build -t vtumca/devsecops-app:${{ github.sha }} .

      # Step 5: Security Scanning with Aquasec Trivy (Shift-Left)
      - name: Run Trivy Vulnerability Scanner
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: 'vtumca/devsecops-app:${{ github.sha }}'
          format: 'table'
          exit-code: '1' # Fails the pipeline if CRITICAL vulnerabilities exist
          ignore-unfixed: true
          severity: 'CRITICAL,HIGH'

      # Step 6: Log in to Docker Hub Registry (Only on Push to Main)
      - name: Login to Docker Hub
        if: github.ref == 'refs/heads/main' && github.event_name == 'push'
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }
```

```
# Step 7: Push Image to Docker Hub
- name: Push Image to Docker Hub
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  run: |
    docker tag vtumca/devsecops-app:${{ github.sha }} vtumca/devsecops-app:latest
    docker push vtumca/devsecops-app:${{ github.sha }}
    docker push vtumca/devsecops-app:latest
```

Key Takeaway Checklist for Exam Day

- [] Understand the 5 NIST Cloud Characteristics: On-demand self-service, Broad network access, Resource pooling, Rapid elasticity, Measured service.
- [] In Kubernetes, remember that a Pod is the smallest deployable compute unit (contains one or more tightly coupled containers).
- [] Memorize the fundamental difference between declarative IaC (Terraform - "What you want") and imperative scripts (Bash - "How to do it").